
Algorithm 1: Powersort($A[1..n]$)

```
 $X$  := stack of runs ;                               //capacity  $\lceil \lg n(n) \rceil + 1$ 
 $P$  := stack of powers ;                               //capacity  $\lceil \lg n(n) \rceil + 1$ 
 $s_1 := 1; e_1 = \text{ExtendRunRight}(A[1], n);$ 
while  $e_1 < n$  do
     $s_2 := e_1 + 1; e_2 := \text{ExtendRunRight}(A[s_2], n);$  //  $A[s_2..e_2]$  next run
     $p := \text{NodePower}(s_1, e_1, s_2, e_2, n);$  //  $P_j$  for node  $j$  between  $A[s_1..e_1]$ 
    and  $A[s_2..e_2]$ 
    while  $P_{\text{top}}() > p$  do
        ; //previous merge deeper in tree than current
         $P_{\text{pop}}();$  //  $\rightarrow$  merge and replace run  $A[s_1..e_1]$  by result
         $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1]);$ 
    end
     $X.\text{push}(A[s_1, e_1]);$ 
     $P.\text{push}(p);$ 
     $s_1 := s_2; e_1 := e_2;$  //Now  $A[s_1..s_1]$  is the rightmost run
end
while  $\neg X.\text{empty}()$  do
     $(s_1, e_1) := \text{Merge}(X.\text{pop}(), A[s_1..e_1]);$ 
end
```
